

Table des matières

1. Objectif de l'application.....	1
2. Modèle métier et diagramme de classes.....	2
Relations.....	2
3. Schéma de la base de données.....	3
4. Architecture de l'application.....	4
5. Principales fonctionnalités développées.....	5
Histoires et personnages (étapes 1 à 3).....	5
Scènes (étape 4).....	6
Recherche, filtrage et statistiques (étape 5).....	7
Jeux de données de démonstration.....	8
6. Tests réalisés.....	8
7. Difficultés rencontrées et solutions apportées.....	9
8. Perspectives d'évolution du projet.....	10

1. Objectif de l'application

StoryForge est une application desktop JavaFX qui aide un utilisateur à concevoir et organiser une histoire de manière structurée, plutôt qu'à travers un simple éditeur de texte libre. Elle permet de centraliser :

- les histoires (titre, auteur, résumé)
- les personnages de chaque histoire (nom, rôle, description)
- les scènes de chaque histoire (titre, lieu, moment, contenu, statut d'avancement, personnages présents), réordonnables par glisser-déposer
- la recherche et le filtrage des scènes par mot-clé, statut et/ou personnage présent
- des statistiques sur l'histoire sélectionnée (nombre de personnages, de scènes, répartition par statut, fréquence d'apparition de chaque personnage)

L'application a été développée progressivement, étape par étape, chaque étape introduisant une nouvelle fonctionnalité ou une nouvelle contrainte technique (persistance, restructuration en couches, nouvelle entité avec ses relations, recherche/filtrage/statistiques, tests automatisés) :

Étape	Contenu
1	Première version JavaFX (histoire + personnages, en mémoire)
2	Ajout de la persistance MySQL

3	Restructuration en couches + SOLID
4	Ajout des scènes (One-To-Many, Many-To-Many)
5	Recherche, filtrage et statistiques
6	Tests unitaires et tests d'intégration

2. Modèle métier et diagramme de classes

Le modèle métier (package model) est composé de quatre classes/énumération :

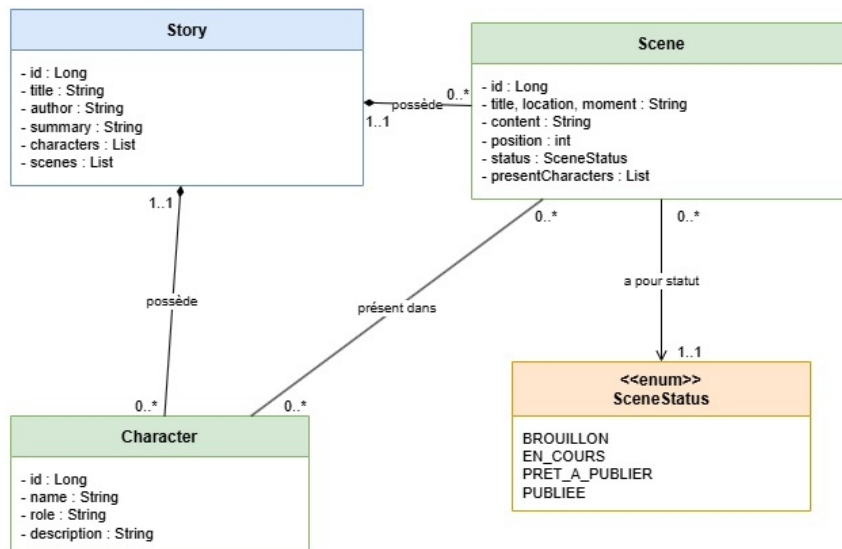
- Story : l'agrégat racine. Possède un titre (obligatoire), un auteur (obligatoire), un résumé (optionnel), une liste observable de Character et une liste observable de Scene. C'est elle qui fait respecter les règles qui dépendent de plusieurs personnages/scènes à la fois : unicité du nom d'un personnage au sein de l'histoire, contiguïté des positions des scènes (toujours 1..N, recalculées automatiquement), appartenance obligatoire au casting pour qu'un personnage soit ajouté à une scène.
- Character : nom (obligatoire), rôle (obligatoire), description (optionnelle). Ne valide que ses propres champs.
- Scene : titre (obligatoire), lieu (optionnel), moment (optionnel), contenu (obligatoire), position (dérivée, jamais saisie), statut (obligatoire), liste observable des personnages présents.
- SceneStatus : énumération à 4 valeurs : Brouillon, En cours, Prêt à publier, Publiée.

Relations

- Story 1 → * Character (une histoire possède plusieurs personnages, un personnage appartient à une seule histoire).

- Story 1 → * Scene (une histoire possède plusieurs scènes, une scène appartient à une seule histoire).
- Scene * ↔ * Character (une scène peut contenir plusieurs personnages, un personnage peut apparaître dans plusieurs scènes), avec la règle métier qu'un personnage présent dans une scène doit obligatoirement appartenir au casting de l'histoire.

Diagramme de classes : StoryForge



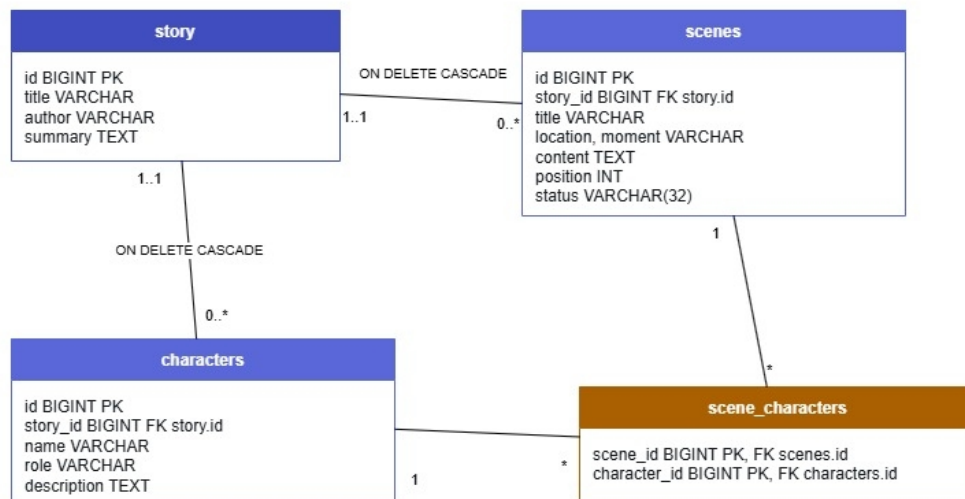
3. Schéma de la base de données

Le schéma SQL (`src/main/resources/db/schema.sql`) comporte quatre tables :

- `story` : `id` (clé primaire), `title`, `author`, `summary`.
- `characters` : `id`, `story_id` (clé étrangère vers `story`, ON DELETE CASCADE), `name`, `role`, `description`. Contrainte `UNIQUE(story_id, name)`, qui reflète en base la règle métier d'unicité déjà vérifiée côté Java.
- `scenes` : `id`, `story_id` (clé étrangère, cascade), `title`, `location`, `moment`, `content`, `position`, `status` (stocké comme le nom textuel de l'énumération, ex. "BROUILLON"). Pas de contrainte `UNIQUE(story_id, position)` : retirée volontairement (voir section 7, difficultés).
- `scene_characters` : table de jonction pour la relation Many-To-Many, clé composite (`scene_id`, `character_id`), deux clés étrangères en cascade.

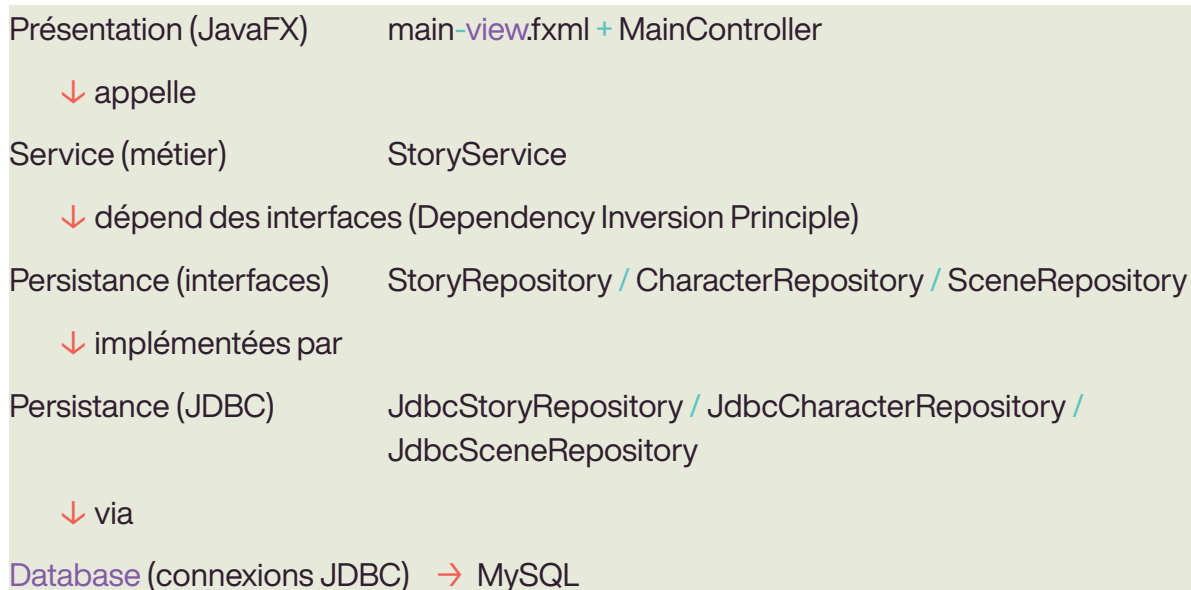
Toutes les suppressions en cascade (ON DELETE CASCADE) sont déléguées à MySQL plutôt que recodées en Java : supprimer une histoire supprime automatiquement ses personnages, ses scènes et les associations scène-personnage.

Schéma de la base de données (MySQL)



4. Architecture de l'application

L'application suit une architecture en 4 couches mise en place à l'étape 3 et étendue ensuite sans jamais être remise en cause :



MainController ne contient aucune logique métier ni SQL : il lit/ écrit les champs de l'interface et appelle StoryService.

StoryService applique d'abord la règle métier (en appelant le modèle), puis répercute le changement en base (en appelant le repository correspondant). Il porte aussi la logique de recherche/filtrage/ statistiques (en mémoire, sans accès base), conformément à la consigne selon laquelle ces traitements doivent rester dans la couche métier.

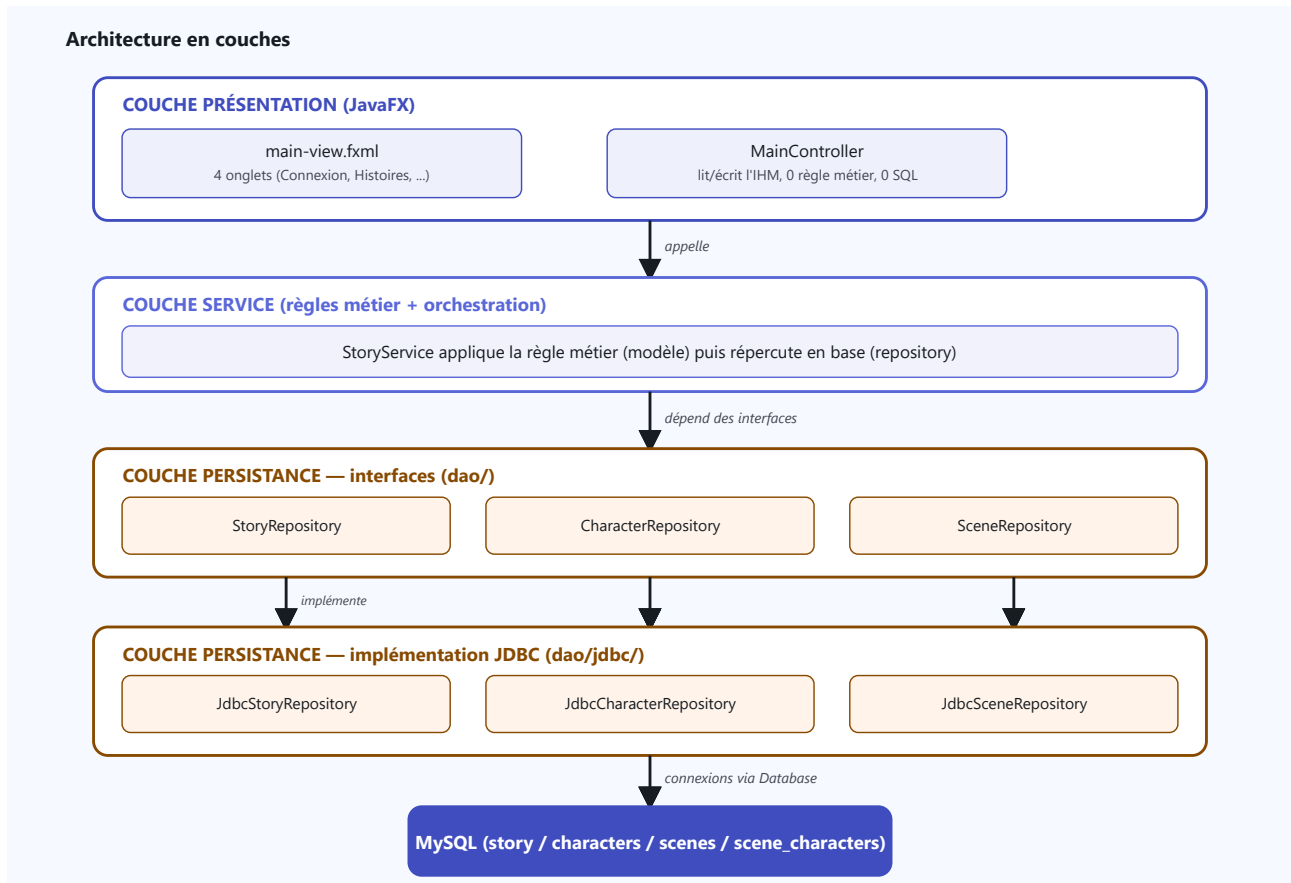
Les interfaces de repository (dao/) découplent StoryService de JDBC : seules les implémentations (dao/jdbc/) connaissent MySQL.

Database centralise l'ouverture des connexions JDBC et la création de la base/des tables.

Principes SOLID appliqués :

Single Responsibility Principle : chaque classe a une seule raison de changer (le modèle change si une règle métier change, les repositories JDBC changent si le SQL change, le service change si l'enchaînement des opérations métier change, le contrôleur change si l'interface change).

Dependency Inversion Principle : StoryService dépend des interfaces StoryRepository / CharacterRepository / SceneRepository, jamais des classes JDBC concrètes. Cela permet de substituer une implémentation en mémoire sans toucher au service, c'est exactement ce que fait StoryServiceTest pour les tests d'intégration (voir section 6).



5. Principales fonctionnalités développées

Histoires et personnages (étapes 1 à 3)

- Création, modification, suppression d'histoires et de personnages.
- Règles métier : titre/auteur d'histoire non vides, résumé optionnel ; nom/rôle de personnage non vides et nom unique dans l'histoire, description optionnelle.
- Persistance MySQL complète (sauvegarde, rechargement, mise à jour, suppression en cascade).

- Écran de connexion permettant de configurer hôte/port/base/utilisateur/mot de passe, de créer la base et ses tables si besoin, et de démarrer le service Windows MySQL directement depuis l'application.

The screenshot shows a web application interface for MySQL connection configuration. At the top, there are four tabs: 'Connexion MySQL' (active), 'Histoires et personnages', 'Scènes', and 'Statistiques'. Below the tabs, the configuration fields are as follows:

- Hôte:** A text input field containing 'localhost'.
- Port:** A text input field containing '3306'.
- Base de données:** A text input field containing 'storyforge'.
- Utilisateur:** A text input field containing 'root'.
- Mot de passe:** A password input field with four dots, indicating it is masked.

Below the password field, there are three buttons:

- 'Se connecter (crée la base si besoin)'
- 'Démarrer le service MySQL (net start)'
- 'Tout supprimer (vide la base)' (highlighted in red)

At the bottom, a status bar indicates 'Connecté à localhost:3306/storyforge'.

Scènes (étape 4)

- Nouvelle entité Scene avec relation One-To-Many vis-à-vis de Story et relation Many-To-Many vis-à-vis de Character.
- Réordonnement des scènes par glisser-déposer dans la liste, la position est toujours recalculée automatiquement (1..N sans trou ni doublon possible).
- Sélection multiple des personnages présents dans une scène, limitée au casting de l'histoire.

Connexion MySQL
Histoires et personnages
Scènes
Statistiques

Recherche / filtrage

Mot-clé (titre, contenu)

Tous les statuts

Tous les personnages

Rechercher

Réinitialiser

Titre
Lieu
Moment
Statut

Brouillon

Scènes de l'histoire sélectionnée (glisser-dé...

1. La découverte du livre
2. Le témoignage de Victor
3. La révélation du secret
4. Recherche dans les archives

Contenu

Personnages présents (les personnages de l'histoire, sélection multiple)

Emma (Étudiante)
Lucas (Journaliste)
Madame Moreau (Bibliothécaire)

Nouvelle

Supprimer

Enregistrer la scène

Recherche, filtrage et statistiques (étape 5)

Recherche par mot-clé (titre et/ou contenu de la scène), filtrage par statut et/ou par personnage présent, directement intégrés à la zone de gestion des scènes (et non dans un écran séparé), avec un bouton de réinitialisation des filtres.

Statistiques affichées dans un onglet dédié : nombre total de personnages, nombre total de scènes, nombre de scènes par statut, nombre d'apparitions de chaque personnage recalculées automatiquement après chaque modification, sans action manuelle de l'utilisateur.

Connexion MySQL
Histoires et personnages
Scènes
Statistiques

Statistiques de l'histoire sélectionnée (mises à jour automatiquement)

Nombre total de personnages : 4
Nombre total de scènes : 4

Scènes par statut :
- Brouillon : 1
- En cours : 2
- Prêt à publier : 1
- Publiée : 0

Apparitions par personnage :
- Emma : 3
- Lucas : 3
- Madame Moreau : 2
- Victor : 2

Jeux de données de démonstration

Deux boutons permettent de charger en un clic un jeu de données complet (histoire, personnages, scènes, associations) : « Légendes arthuriennes » et « Le Secret de la Bibliothèque » (ce dernier reprenant exactement le scénario de démonstration imposé par l'énoncé du projet). Les deux scénarios partagent le même moteur de chargement générique (loadScenario), seules les données diffèrent.

6. Tests réalisés

Mis en place à l'étape 6, avec ajout explicite du plugin maven-surefire-plugin (voir section 7) pour que mvn test exécute réellement les tests JUnit 5.

Tests unitaires (src/test/.../model)

Portent sur le modèle seul, sans aucune dépendance externe :

- StoryTest : rejet d'un titre/auteur vide, unicité du nom d'un personnage (insensible à la casse), suppression d'un personnage qui le retire aussi de ses scènes, refus d'ajouter à une scène un personnage hors casting, contiguïté des positions de scène après suppression et déplacement.
- SceneTest : rejet d'un titre/contenu vide, rejet d'un statut nul.

Tests d'intégration (src/test/.../service/StoryServiceTest)

Font collaborer StoryService avec de vrais objets du modèle et trois repositories en mémoire (InMemoryStoryRepository, InMemoryCharacterRepository, InMemorySceneRepository), rendus possibles par le Dependency Inversion Principle (section 4) donc sans dépendre d'une base MySQL réelle :

- fullWorkflowPersistsAndLinksCorrectly : création d'une histoire, d'un personnage et d'une scène, association des deux, vérification que le faux repository de scènes reçoit bien les appels d'ajout/retrait de lien.
- searchScenesFiltersByKeywordAndStatus : filtrage par mot-clé seul, par statut seul, par personnage seul et par combinaisons, y compris la vérification que le mot-clé ne porte que sur le titre/contenu et pas sur le lieu (conformément à l'énoncé).
- countTotalCharactersAndScenes, countScenesByStatusCountsEveryStatus..., countSceneAppearancesByCharacter : couvrent les méthodes de statistiques de l'étape 5.

Résultat actuel : 12 tests, 0 échec (.\\mvnw.cmd test).


```

WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.storyforge:StoryForge >-----
[INFO] Building StoryForge 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ StoryForge ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 3 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ StoryForge ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ StoryForge ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\Users\William\IdeaProjects\StoryForge\src\test\resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) @ StoryForge ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- maven-surefire-plugin:3.2.5:test (default-test) @ StoryForge ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.storyforge.storyforge.model.SceneTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.072 s -- in com.storyforge.storyforge.model.SceneTest
[INFO] Running com.storyforge.storyforge.model.StoryTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.026 s -- in com.storyforge.storyforge.model.StoryTest
[INFO] Running com.storyforge.storyforge.service.StoryServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.042 s -- in com.storyforge.storyforge.service.StoryServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0

```

7. Difficultés rencontrées et solutions apportées

- **Démarrage du service MySQL nécessitant les droits administrateur** : net start échoue sans élévation. Solution : relance de la commande dans un cmd élevé via PowerShell (Start-Process -Verb RunAs, ce qui déclenche l'invite UAC Windows), avec récupération du résultat via un fichier temporaire (la capture directe du flux standard n'étant pas possible avec un process lancé en élévation), décodé en CP850 (codepage console française) pour préserver les accents.
- **Contrainte d'unicité de position devenue un obstacle au glisser-déposer** : la première version imposait UNIQUE(story_id, position) en base. Or réordonner deux scènes adjacentes fait transitoirement exister deux fois la même position le temps des UPDATE intermédiaires. Solution : la position d'une scène est devenue une valeur entièrement dérivée de son rang dans la liste Java (Story.renumberScenes), jamais saisie ni vérifiée pour elle-même ; la contrainte UNIQUE a été retirée du schéma, avec une migration automatique (ALTER TABLE ... DROP INDEX) pour les bases déjà créées avant ce changement, elle-même bloquée une première fois par MySQL (erreur 1553, l'index composite servant de support à une clé étrangère), résolue en créant d'abord un index simple sur story_id.
- **Glisser-déposer cassé par le filtrage des scènes** : une fois la recherche/filtrage de l'étape 5 ajoutée, l'index de cellule affiché ne correspondait plus à l'index réel dans Story.getScenes() dès qu'un filtre masquait des scènes. Solution : le glisser-déposer retrouve désormais la position réelle de la scène cible via Story.getScenes().indexOf(...) plutôt que via l'index de cellule.

8. Perspectives d'évolution du projet

- **Chapitres/épisodes** : l'énoncé mentionne la possibilité de regrouper les scènes en chapitres ou épisodes ; cette entité supplémentaire (One-To-Many vis-à-vis de Scene) n'a pas été développée mais s'intégrerait dans l'architecture existante sans la remettre en cause.
- **Export** : export des histoires (résumé, personnages, scènes) vers un format externe (PDF, Markdown) pour partage ou impression.
- **Annulation/historique** : ajout d'un mécanisme d'annulation (undo) pour les actions de suppression, actuellement irréversibles côté base (à l'exception du bouton « Tout supprimer », déjà protégé par une confirmation).
- **Multi-utilisateur** : la persistance actuelle suppose un seul utilisateur à la fois ; une gestion de comptes/permissions serait nécessaire pour un usage partagé.